

SABO: Safe and Aggressive Bayesian Optimization for Automatic Legged Locomotion Controller Tuning

Haobo Zhang^{1*}, Zhiyong Yu^{1*}, Hengquan Guo¹, Yuning Jiang², Yizheng Zhang³, Xin Liu¹, Yuanming Shi¹

Abstract—This paper addresses the challenge of optimizing controller parameters in complex robotic systems under safety constraints. We propose Safe and Aggressive Bayesian Optimization (SABO) algorithm, which leverages Gaussian processes (GPs) to model both reward and constraint functions while using an adaptive penalty to balance the tradeoff of reward and cost. Compared to the state-of-the-art algorithm, GOSAFEOPT, which relies on prior system knowledge—such as an initial safe set and the Lipschitz constant of cost functions—to construct a safe and conservative decision set, SABO synthesizes a single surrogate function by adaptively penalizing the cost functions, and encourages safe but aggressive exploration. We implement SABO in both simulations and hardware experiments on a quadruped robot. The results show that it outperforms GOSAFEOPT, offering a safer, more sample-efficient, and robust solution for bridging the sim-to-real gap in dynamic environments. The code and video can be found at <https://sabo-6.github.io>.

I. INTRODUCTION

In the field of control systems, particularly in the context of complex systems such as quadruped robots, the discrepancy between simulation and real-world conditions necessitates the critical adjustment of controller parameters [1]. Simulation typically assumes idealized physical conditions and precise dynamic models. However, in practical applications, robots will encounter various uncertainties, such as sensor inaccuracies, nonlinear actuator behaviors, environmental variations, and discrepancies between the model and the actual system [2–4]. Due to the complexity of real-world environments, these factors cannot be directly modeled in simulations, making it impossible to establish an error function precisely. Consequently, control parameters optimized in simulations often fail to perform effectively in real-world applications, potentially leading to unstable robot behavior [5]. Manual parameter tuning in complex robotic control systems is generally inefficient. As the complexity of a system increases, particularly in terms of factors such as the degrees of freedom in joints, the task of manually adjusting controller parameters becomes increasingly challenging and time-consuming. The effectiveness of manual tuning is often contingent upon the operator’s experience, which can lead to variable outcomes. Consequently, techniques such as optimization algorithms and machine learning are progressively

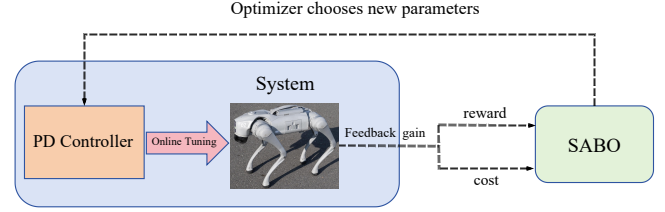


Fig. 1. Overview of the algorithm. The PD controller regulates the quadruped robot’s motion, while the system provides feedback on performance. SABO uses this feedback to evaluate reward (e.g., accuracy) and cost (e.g., energy or safety risks) and iteratively adjusts the controller’s parameters. This process ensures adaptive tuning while maintaining safety constraints, allowing the robot to improve performance in dynamic environments.

replacing traditional manual approaches. These automated methods offer more efficient and precise exploration of the parameter space, along with enhanced adaptability and scalability.

In general, the pursuit of optimal controller parameters is critical for ensuring stable and efficient robot locomotion in the presence of various real-world uncertainties, thereby achieving the desired control performance. To this end, if the functional relationship between controller parameters and real-world performance were known, optimal parameters could be obtained by solving an optimization problem. However, due to the complexity and unpredictability of real-world environments—such as friction, noise interference, and other dynamic factors—this relationship remains unknown, presenting a black-box challenge. In response to this issue, Bayesian optimization has emerged as a promising data-driven methodology, offering enhanced sample efficiency over traditional grid search and gradient-based approaches [6]. This probabilistic framework operates under the theoretical foundation that the objective function lies in a reproducing kernel Hilbert space (RKHS) with a bounded norm, enabling the relationship between parameters and performance to be modeled through Gaussian processes. Such a framework not only improves the efficiency of the optimization process but also allows for the effective handling of uncertainty and the exploration of complex, high-dimensional parameter spaces. A notable example is the application of contextual GOSAFEOPT [7], a safe Bayesian optimization method used for tuning controller parameters on a quadruped robot [8]. This method offers safety and sample complexity guarantees compared with conventional methods, making it particularly effective at addressing the sim-to-real gap in robotic control. By integrating safety constraints into the optimization process, GOSAFEOPT models both the reward and constraint functions as Gaussian

*Equal contribution.

¹H. Zhang, Z. Yu, H. Guo, X. Liu and Y. Shi are with the School of Information Science and Technology, ShanghaiTech University. Emails: {zhanghb2023, yuzhy2023, guohq, liuxin7, shiym}@shanghaitech.edu.cn. ²Y. Jiang is with Automatic Control Laboratory, EPFL. Email: yuning.jiang@ieee.org. ³Y. Zhang is with the Tencent Robotics X. Email: yizhengzhang@tencent.com.

processes, ensuring safety while exploring the parameter space. Specifically, the algorithm starts from a known safe initial set, expands the safe set through local exploration by assuming a known Lipschitz constant for the constraint function, and then proceeds to conduct global exploration by verifying boundary conditions. While the algorithm demonstrates strong theoretical capabilities in identifying globally optimal and safe solutions, its practical implementation faces significant challenges due to the reliance on prespecified parameters, i.e. the initial safe set and Lipschitz constant. In dynamic environments or when there are variations in the robot’s hardware or operating conditions, these parameters may become ineffective or outdated, leading to suboptimal performance or safety violations. As a result, to ensure the algorithm operates successfully, these parameters need to be adjusted accordingly, which contradicts the goal of achieving automatic parameter tuning.

Our contributions. Inspired by [9], we propose and apply the Safe and Aggressive Bayesian Optimization (SABO) algorithm to the problem of automatic tuning of parameters. Unlike GOSAFEOP’s *conservative* method of expanding the safe set, SABO uses an *aggressive*, penalty-based framework that integrates the reward and constraint functions and adaptively to be safe. This approach reduces the computational complexity associated with finding the safe set at each iteration, thereby obviating the need for prior knowledge of the Lipschitz constant—a parameter that is often impractical to ascertain. Furthermore, SABO does not require an initial safe set to guide exploration, enhancing its practicality and robustness. Beyond theoretical advantages, we have evaluated SABO in both simulation and hardware experiments and compared its performance against GOSAFEOP. Specifically, we implement SABO on the quadruped robot *Unitree Go2* [10] and conduct a series of tests across various gait patterns. The results demonstrate that SABO outperforms GOSAFEOP in terms of safety and robustness, effectively handling dynamic environmental variations while maintaining stable locomotion. Notably, SABO exhibits sample efficiency, identifying near-optimal gait parameters in fewer iterations in experiments, whereas GOSAFEOP fails to converge to a stable solution. In hardware experiments, the controller, when integrated with our algorithm, consistently avoids unsafe interactions with the robot, underscoring the robustness of SABO in addressing the sim-to-real gap.

II. RELATED WORK

a) Bridging the sim-to-real gap: Due to the significant discrepancies between simulation environments and the real world in terms of physical modeling, friction coefficients, and environmental dynamics, a substantial body of research focuses on addressing the sim-to-real gap. Various methods have been proposed, such as dynamics randomization [11], domain randomization [12], and system identification [13], among others. These approaches aim to simulate the complexities of the real world within the simulation environment, ensuring that model parameters trained in simulation remain

effective in real-world scenarios. For instance, Ensemble-CIO [11] (a dynamics randomization method) introduces a novel dynamic motion planning framework capable of handling complex contact and friction issues while leveraging ensemble learning to enhance policy performance in physical environments. Domain randomization [12] seeks to randomize rendering parameters in the simulation (such as textures, lighting, camera positions, etc.), by which the model can be exposed to a diverse range of environments during training. This approach aims to enable the model to adapt to various potential real-world environments, thereby enhancing its generalization ability in real-world applications. [13] uses physics engines for system identification, which reduces the sim-to-real gap and thereby improves the policy learning process. Although these methods provide significant help with sim-to-real transfer, when deploying in real environments, it is crucial to ensure that the model can handle the complexities and uncertainties of the real world. Therefore, online identification remains an important step for the confirmation and refinement of the model.

b) Safe Bayesian methods for automatically tuning controller parameter: In contrast to simulating real-world conditions to adjust parameters, a more direct approach is to use Bayesian optimization methods to tune parameters directly on hardware. A notable example is the application of Bayesian optimization to bipedal robots [6], where the algorithm adjusts controller parameters to achieve higher speed. The study also compares other search methods with Bayesian optimization, demonstrating the inherent advantages of Bayesian methods in parameter tuning. However, this approach does not consider safety constraints, which are critical to ensuring the robot’s safety during parameter adjustments in real-world scenarios. In contrast, SAFEOP [14] and its variants [15, 5, 7, 8] address this limitation by iteratively expanding a safe set during the search process, ensuring that the algorithm identifies safe and efficient controller parameters. Specifically, SAFEOP assumes a safe initial policy and a known Lipschitz constant to expand the safe set around the initial point, though it only guarantees convergence to a local optimum. GOSAFE [15] extends SAFEOP by incorporating global exploration, enabling convergence to a global optimum but struggling with high-dimensional state spaces. Furthermore, GOSAFEOP [7] leverages the Markov property of the system, combining SAFEOP and GOSAFE to identify global optima in high-dimensional state spaces. Recently, SAFEOP-type methods [5, 8], including contextual GOSAFEOP [8], have been applied to tuning controller parameters for quadruped robots, demonstrating some success.

However, as previously discussed, SAFEOP-type methods require numerous predefined parameters, which limits their practicality in real-world applications. In this work, we apply a more practical algorithm, SABO, for parameter tuning. Through experiments, we demonstrate that our algorithm can identify optimal parameters for quadruped robots while ensuring safety.

III. PROBLEM STATEMENT

In control systems, feedback-based control strategies are critical for ensuring robustness and adaptability in dynamic and uncertain environments. The control input $\mathbf{u}$ in such systems is typically determined through a feedback function $\pi_{\mathbf{x}}$, parameterized by $\mathbf{x}$ within the domain $\mathcal{X}$. This function takes into account the ideal control input $\mathbf{u}^*$, the desired state $\mathbf{s}^*$, and the current system state $\mathbf{s}$. Specifically, the control system can be expressed as:

$$\mathbf{u} = \pi_{\mathbf{x}}(\mathbf{u}^*, \mathbf{s}^*, \mathbf{s}). \quad (1)$$

This feedback strategy allows the system to adjust its actions in real-time, correcting errors and stabilizing performance based on deviations between the current state and the target state. A widely used feedback-based strategy is PD control, where the controller parameter $\mathbf{x}$ is represented by $\mathbf{k}_p$ and $\mathbf{k}_d$ in this context. In this paper, we focus on PD control without loss of generality due to its commonality and popularity. The formulation of PD control law is given by:

$$\mathbf{u} = \mathbf{u}^* + \mathbf{k}_p(\bar{\mathbf{s}}^* - \bar{\mathbf{s}}) + \mathbf{k}_d(\dot{\bar{\mathbf{s}}}^* - \dot{\bar{\mathbf{s}}}), \quad (2)$$

where $\mathbf{k}_p \in \mathbb{R}^{d_u \times d_s}$ and $\mathbf{k}_d \in \mathbb{R}^{d_u \times d_{\dot{s}}}$. The term $\bar{\mathbf{s}}^* - \bar{\mathbf{s}}$ denotes the position error and $\dot{\bar{\mathbf{s}}}^* - \dot{\bar{\mathbf{s}}}$ represents the velocity error. This control strategy is often employed to perform various tasks, such as reference tracking, where the goal is to minimize the error between the current and desired states by adjusting the policy parameters $\mathbf{x}$. To achieve this in a safe pattern, we can formulate it as a constrained optimization problem:

$$\max_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad \text{s.t.} \quad g(\mathbf{x}) \leq 0. \quad (3)$$

Here, $f(\mathbf{x}) : \mathcal{X} \rightarrow \mathbb{R}$ represents the reward function, whose value is evaluated on the system, and $g(\mathbf{x}) : \mathcal{X} \rightarrow \mathbb{R}$ is the constraint function, which captures safety-critical parameters that the controller should avoid. The objective is adjusting the parameter $\mathbf{x}$ to solve this optimization problem. However, in the real system, the exact formulations of f and g are unknown, so the problem is considered a black box fashion.

IV. SAFE BAYESIAN ALGORITHM

Gaussian Process Regression (GPR) [16] is a sample-efficient approach to solving problem (3). For the unknown function, GPR consistently samples new decisions, updates the function's confidence level, and finally converges to an optimal solution. Specifically, the Gaussian model of reward function $f(\mathbf{x})$ over domain $\mathbf{x}$ is defined as $\Theta(\mu^f(\mathbf{x}), k^f(\mathbf{x}, \mathbf{x}'))$, where $\mu^f(\mathbf{x})$ represents the mean and $k^f(\mathbf{x}, \mathbf{x}')$ is the covariance function. After $t-1$ interactions with the environment, the Gaussian process updates its posterior distribution based on the past set of $t-1$ decisions and corresponding feedback, $(\mathbf{x}_i, r_i)_{i=1}^{t-1}$. The posterior mean $\mu^f(\mathbf{x})$, covariance $k^f(\mathbf{x}, \mathbf{x}')$, and variance $\sigma_t^f(\mathbf{x})$ in GP are

computed as follows:

$$\begin{aligned} \mu_t^f(\mathbf{x}) &= k_t^f(\mathbf{x})^\top (V_t^f(\lambda))^{-1} r_{1:t}, \\ k_t^f(\mathbf{x}, \mathbf{x}') &= k^f(\mathbf{x}, \mathbf{x}') - k_t^f(\mathbf{x})^\top (V_t^f(\lambda))^{-1} k_t^f(\mathbf{x}'), \\ \sigma_t^f(\mathbf{x}) &= \sqrt{k_t^f(\mathbf{x}, \mathbf{x})}, \end{aligned}$$

where $[K_t^f]_{(i,j)} = k^f(\mathbf{x}_i, \mathbf{x}_j)$, $i, j \in \{1, \dots, t-1\}$, $V_t^f(\lambda) = K_t^f + \lambda I$, $\lambda = 1 + 3/T$, $r_{1:t} = [r_1, \dots, r_{t-1}]$, and $k_t^f(\mathbf{x}) = [k^f(\mathbf{x}_1, \mathbf{x}), \dots, k^f(\mathbf{x}_{t-1}, \mathbf{x})]^\top$. Similarly, the GP model for the constraint function $g(\mathbf{x})$ follows the same structure and updates. The kernel functions $k^f(\mathbf{x}, \mathbf{x}')$ and $k^g(\mathbf{x}, \mathbf{x}')$, which measure the similarity between two data points, are designed through careful selection and can be different across GPs. Further details on the impact of the choice of kernel function can be referred to [6]. In this work, we use the popular square exponential (SE) kernel, $k_{SE}(\mathbf{x}, \mathbf{x}') = \exp\left(\frac{-\|\mathbf{x} - \mathbf{x}'\|^2}{2u^2}\right)$, where $u > 0$ is a hyperparameter.

The primary goal of safe Bayesian optimization is to find the optimal solution for the black-box problem with as few samples as possible while balancing the trade-off between reward and cost. In the GOSAFEOP approach, the focus is on expanding the safe set and identifying the maximum reward within it. However, this method relies on prior knowledge of the Lipschitz constant of the constraint function and is computationally expensive, potentially hindering real-time application. In contrast, inspired by [9], we adopt a penalty-based decision-making approach, Safe and Aggressive Bayesian Optimization (SABO), that offers low computational complexity and is more suitable for real-world systems. The algorithm is outlined below. In general, SABO maintains a learning strategy UCB/LCB [17] to estimate the reward optimistically and cost functions pessimistically and subsequently selects the decision that maximizes a surrogate function, which properly balances the reward and cost.

SABO Algorithm

Initialization: $Q_1 = 1$ and $\eta_t = \sqrt{t}$. Gaussian process model (Θ_1^f, Θ_1^g) .

For $t = 1, \dots, T$,

- **Estimate reward and cost functions:**

$$\hat{f}_t(\mathbf{x}), \check{g}_t(\mathbf{x}) = \text{UCB/LCB}(\Theta_t^f, \Theta_t^g)$$

- **Decide next parameter:** choose $\mathbf{x}_t$ such that

$$\mathbf{x}_t = \arg \max_{\mathbf{x} \in \mathcal{X}} \hat{f}_t(\mathbf{x}) - Q_t \check{g}_t^+(\mathbf{x}) \quad (4)$$

- **Receive feedback:** reward $r_t(\mathbf{x}_t)$ and cost $c_t(\mathbf{x}_t)$.
- **Update the penalty:**

$$Q_{t+1} = \max(Q_t + c_t^+(\mathbf{x}_t), \eta_t). \quad (5)$$

- **Update GP model:**

$$\Theta_{t+1}^f, \Theta_{t+1}^g = \text{UCB/LCB}\left(\Theta_t^f, \Theta_t^g, \{\mathbf{x}_t, r_t, c_t\}\right). \quad (6)$$

The key concept of SABO lies in penalty-based decision-making. The algorithm selects decisions based on the penalty Q_t , which is multiplied by the estimated constraint function $\tilde{g}_t(\mathbf{x})$, striking a balance between reward and cost. The penalty is updated in correspondence with the maximization of hard constraint violations and the parameter $\eta_t = \sqrt{t}$, which adaptively controls the aggressiveness of decision-making at each iteration. The Gaussian process models are then updated based on the decisions and feedback from the system. The advantage of SABO lies in its ability to avoid alternating between local exploration and global exploration, thus adaptively finding the global optimum. This eliminates the need for prior knowledge of several parameters, unlike in GOSAFEOPT.

The estimators of the reward and cost function, known as GP-UCB/LCB, are defined as follows:

$$\begin{aligned}\hat{f}_t(\mathbf{x}) &= \mu_t^f(\mathbf{x}) + \beta_t^f \sigma_t^f(\mathbf{x}), \\ \tilde{g}_t(\mathbf{x}) &= \mu_t^g(\mathbf{x}) - \beta_t^g \sigma_t^g(\mathbf{x})\end{aligned}$$

where $\hat{f}_t(\mathbf{x})$ estimates the reward function optimistically, and $\tilde{g}_t(\mathbf{x})$ estimates the constraint function pessimistically. β_t^f and β_t^g are parameters to control the exploration, and we specify them in our experiment section.

Theorem 1: Let $\mathbf{x}^*$ be the optimal solution to problem (3) and $\mathbf{x}_t$ is the decision that SABO makes at round t . As $T \rightarrow \infty$, $\mathbf{x}_T$ converges to $\mathbf{x}^*$, and the averaged constraint violation satisfies $\sum_{t=1}^T g^+(\mathbf{x}_t)/T \rightarrow 0$.

This theorem extends the main result in [9], while SABO analyzes the error bound of GP-UCB/LCB and ultimately establishes the algorithm's convergence. In general, *Theorem 1* guarantees that SABO can progressively approach the optimal solution during the optimization process while maintaining high stability and reliability under constraint conditions. This makes it particularly suitable for scenarios that require safe and sample-efficient approaches, such as tuning controller parameters in real-world robotic systems.

V. EXPERIMENTS

In this section, we present the experimental results evaluating the performance of our method, SABO, for tuning the trot PD controller's parameters in both simulation and real-world environments using safe Bayesian optimization. The control framework of the quadruped robot consists of Nonlinear Model Predictive Control (NMPC), which outputs the desired state, and Whole Body Control (WBC), which outputs the desired torque. The robot's state is estimated using a State Estimation module [18]. The update frequencies for NMPC, WBC, and State Estimation are 30 Hz, 500 Hz, and 500 Hz, respectively. To demonstrate the advantages of our algorithm, we compare its performance to baseline methods in terms of objective value, constraint violation, and convergence speed. The objective and constraints used in the experiments are as follows:

$$\begin{aligned}f(\mathbf{x}) &= - \sum_{r_i > 0} \|\bar{\mathbf{s}}^*(r_i) - \bar{\mathbf{s}}(r_i, \mathbf{x})\|_2^2, \\ g(\mathbf{x}) &= \max_{r_i > 0} \|\bar{\mathbf{s}}^*(r_i) - \bar{\mathbf{s}}(r_i, \mathbf{x})\|_2^2 - v,\end{aligned}$$

where r_i denotes time in the real world ($r_{i+1} - r_i = 2$ seconds), $\bar{\mathbf{s}}^*(r)$ and $\bar{\mathbf{s}}(r, \mathbf{x})$ represent the desired and actual joint-level states (i.e., joint angles and speeds) of the system, respectively. v is the error threshold, meaning that the state error should not exceed this value throughout the entire duration.

A. Experiments Detail

In our experiments, we used a computer equipped with an i7-12700H CPU and a 3080Ti GPU to run the Gazebo simulator. The hardware platform consists of the *Unitree Go 2* robot [10]. As a baseline, we implemented the GOSAFEOPT method [7], which automates the optimization process while ensuring safety during the learning phase by confining the system to a "safe" region. This method leverages contextual information, such as the gait angle, to dynamically update the confidence region and alternates between local and global search strategies to efficiently converge to the global optimum. The parameters of the PD controller, denoted as kp

TABLE I
GAIT PARAMETERS.

Gait Pattern	Duty cycle	Phase Offsets
Flying trot	0.5	[0.5, 0, 0, 0.5]
Trot	0.5	[0.5, 0, 0, 0.5]
Pace	0.5	[0, 0, 0.5, 0.5]
Standing trot	0.5	[0.5, 0, 0, 0.5]
Static walk	0.6	[0, 0.25, 0.5, 0.75]

TABLE II
PARAMETERS OF SABO AND GOSAFEOPT.

Parameter	SABO	GOSAFEOPT	GOSAFEOPT(HD)
lengthscale		0.1	0.5
lower bound		[0.5,0.5]	[1,1]
upper bound			[9,9]
B_f	7	~	~
B_g	2	~	~
R_f	0.1	~	~
R_g	0.1	~	~
p	0.05	~	~
Q	1	~	~
Interior lower bound	~	0.2	0.05
Marginal lower bound	~	0.6	0.1
Θ_p	~		1
Θ_g	~		1
ω	~		0.9

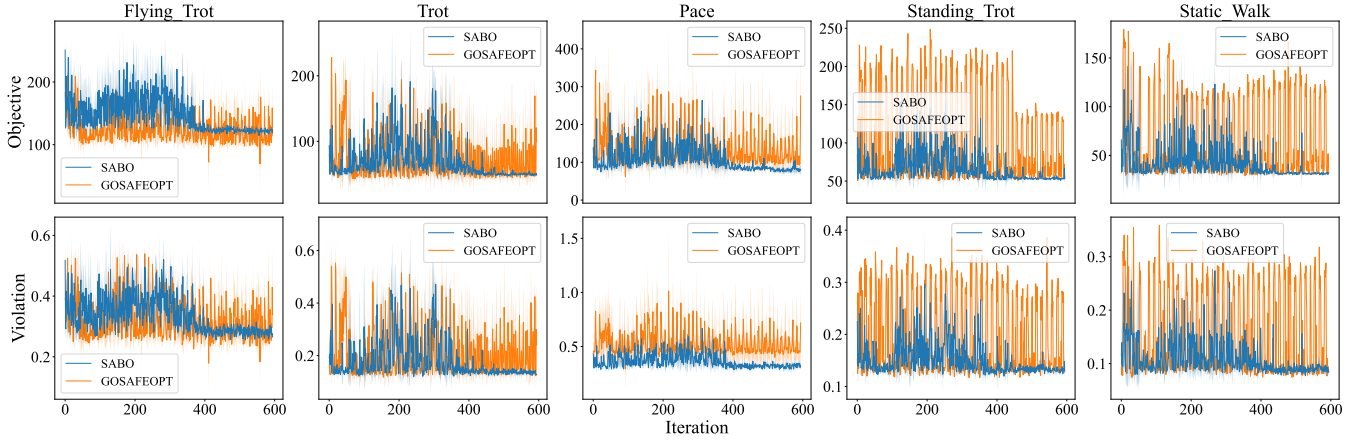


Fig. 2. **Simulation results of SABO and GOSAFEOPT.** We compared the objective and violation performance of SABO and GOSAFEOPT across flying, trotting, and walking gaits. For both algorithms, we applied 600 iterations for each gait, using the same set of parameters as mentioned previously, without any changes. During the first 400 iterations, no clear pattern emerged. However, after 400 rounds, SABO had gathered sufficient data to converge more quickly. Due to the complexity of the pace gait, there is a notable performance gap between SABO and GOSAFEOPT.

and kd , are both vectors in $\mathbb{R}^{12}$. Although low-dimensional Bayesian optimization can be extended to higher-dimensional problems, the large solution space in high-dimensional settings presents a significant challenge, making it difficult to apply a simple Bayesian optimizer directly [19]. A promising approach is to project the high-dimensional space into a lower-dimensional subspace, [20–22]. In order to simplify the experiment, we chose to optimize a lower-dimensional problem. We set the ranges for $k_{p_i} \in \mathbf{k}_p$ and $kd_i \in \mathbf{k}_d$ to $[1, 9]$, respectively. The lengthscale (ls) was set to 0.1 for the simulator and 0.5 for the hardware in order to reduce convergence time. A threshold of 0.3 was used for the experiments. More details on the parameters used are provided in Tables I and II, where GOSAFEOPT(HD) denotes the parameters used in the hardware experiments. The training process consisted of 600 rounds in the simulator and 300 rounds on the hardware. Each training session of 600 rounds required approximately one hour of wall-clock time.

B. Simulation Results

In this section, we begin by examining a low-dimensional task in which all 12 parameters of $\mathbf{k}_p$ and $\mathbf{k}_d$ are set to the same value (e.g., $\mathbf{k}_p = [3, \dots, 3]^T$), effectively reducing the problem to a 2-dimensional one. This simplification allows for a clearer comparison between our method (SABO) and GOSAFEOPT. We evaluate three gait patterns—flying trot, trot, and pace—to highlight the advantages of our algorithm in terms of both convergence speed and objective performance, as illustrated in Fig. 2. The reason for the initial value being 0 is that the gait is in the Stance pattern. While environmental noise causes some fluctuations in the objective values of SABO and GOSAFEOPT, we observe that in Fig. 2, the objective and violation metrics of SABO align closely with the best performance achieved by GOSAFEOPT during the convergence phase. This suggests that SABO performs comparably to GOSAFEOPT in terms of final objective value and violation. The violations of GOSAFEOPT are many

times larger than the threshold 0.3 that we set during training. Moreover, SABO demonstrates faster convergence, requiring fewer parameter adjustments. Specifically, during the first 350 rounds, SABO explores the system more extensively to gather information. After approximately 350 rounds, we observe a sharp decline in the objective, signaling that sufficient information has been acquired and the optimizer is converging effectively. This behavior is also evident in the trot and pace patterns. Based on the simulator experiments, we conclude that SABO outperforms GOSAFEOPT in both convergence speed and objective performance.

C. Hardware Results

We successfully deployed the optimizer on the *Unitree Go 2* robot to demonstrate SABO’s sim-to-real transfer capabilities. To reduce convergence time, we set the ls to 0.5, narrowing the solution space. The optimizer was initialized with random parameters, and training was conducted for about 400 rounds to analyze convergence and optimization performance. The convergence of our algorithm is depicted in Fig. 3. After approximately 250 exploration rounds, our algorithm converges to several optimal points. Due to the larger ls , the optimization problem becomes easier, resulting in multiple points with similar objective values. Consequently, while the algorithm’s objective converges, the solution fluctuates between several near-optimal points. Next, we compare the optimization performance of our algorithm with the GOSAFEOPT, as shown in Table III. Specifically, we use the deviations after a given time to evaluate the accuracy of $\mathbf{k}_p$ and $\mathbf{k}_d$. To make the results clearer, we set the task to maintain the original position and minimize the deviations caused by inappropriate $\mathbf{k}_p$ and $\mathbf{k}_d$. In trot and flying trot gait patterns, SABO’s solution results in a smaller deviation compared to the GOSAFEOPT, indicating that, with fewer hyperparameter adjustments, the $\mathbf{k}_p$ and $\mathbf{k}_d$ values tuned by our algorithm lead to better performance. This improvement can be attributed to GOSAFEOPT’s reliance

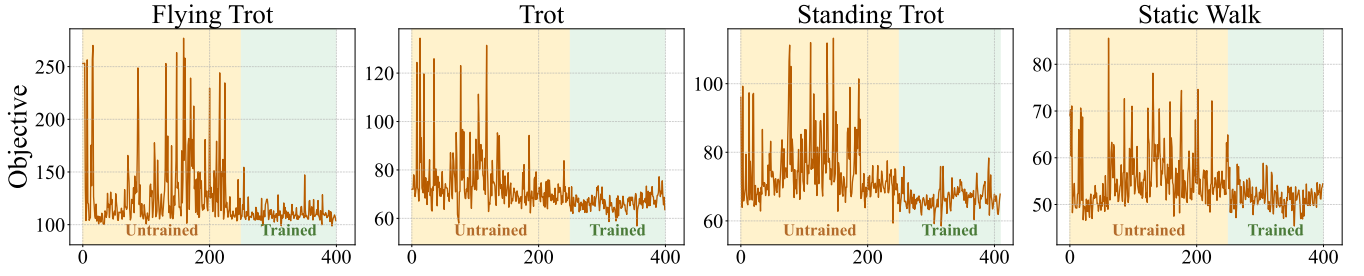


Fig. 3. **SABO combines flying trot, trot, standing trot and static walk on hardware.** We use the same parameters for both the simulator and hardware, with the exception of $ls = 0.5$. The deviation and objective exhibit poor performance before training due to suboptimal values of k_p and k_d . After approximately 250 training rounds, the optimizer identifies better parameters, leading to a reduction in both deviation and objective.



Fig. 4. **Safety compare of pace gait between SABO and GOSAFEPT.** Due to the challenging nature of the pace gait, GOSAFEPT may fail within the first 50 iterations, whereas SABO is able to continue the optimization process.

TABLE III

SABO AND GOSAFEPT DEVIATIONS IN 60 SECONDS FOR TWO GAITS WHILE STANDING STILL (IN CENTIMETERS)

	Flying trot	Standing trot	Trot	Static walk
SABO	80.85	44.5	42.34	55.2
GOSAFEPT	128.12	49.7	66.55	57.3

on extensive knowledge of specific conditions, such as the initial safe set and the Lipschitz constant, which are less critical for SABO. Additionally, during the training process for the pace gait, both SABO and GOSAFEPT experienced failures, occasionally falling to the ground. However, SABO maintained a longer safe training duration compared to GOSAFEPT. Specifically, GOSAFEPT failed after the first 50 iterations, while SABO continued training. Given the inherent difficulty of maintaining balance in the pace gait, GOSAFEPT’s sensitivity to parameter selection may have contributed to its premature failure.

VI. CONCLUSION

We present the Safe and Aggressive Bayesian Optimization (SABO) algorithm to the problem of tuning controller parameters in robotic systems under safety constraint. SABO models both reward and constraint functions using Gaussian processes, integrating them through an adaptive penalty to ensure that safety constraints are consistently satisfied. This adaptive penalty-based framework enhances SABO’s sample efficiency and practical applicability compared to the state-of-the-art algorithm, (GOSAFEPT). Specifically, SABO eliminates GOSAFEPT’s need for prior knowledge of the initial safe set and the Lipschitz constant, which are

typically challenging to obtain in real-world environments. We implement SABO in both simulations and hardware experiments, and the results demonstrate that SABO outperforms GOSAFEPT across a range of scenarios, offering a more sample efficient and robust solution in bridging the sim-to-real gap in safety-critical robotics.

REFERENCES

- [1] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: a survey,” in *2020 IEEE symposium series on computational intelligence (SSCI)*. IEEE, 2020, pp. 737–744.
- [2] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.
- [3] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, “Sim-to-real: Learning agile locomotion for quadruped robots,” *arXiv preprint arXiv:1804.10332*, 2018.
- [4] J. Rothfuss, B. Sukhija, L. Treven, F. Dörfler, S. Coros, and A. Krause, “Bridging the sim-to-real gap with bayesian inference,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 10 784–10 791.
- [5] F. Berkenkamp, A. Krause, and A. P. Schoellig, “Bayesian optimization with safety constraints: safe and automatic parameter tuning in robotics,” *Machine Learning*, vol. 112, no. 10, pp. 3713–3747, 2023.
- [6] R. Calandra, A. Seyfarth, J. Peters, and M. P. Deisenroth, “Bayesian optimization for learning gaits under uncertainty: An experimental comparison on a dynamic bipedal walker,” *Annals of Mathematics and Artificial Intelligence*, vol. 76, pp. 5–23, 2016.
- [7] B. Sukhija, M. Turchetta, D. Lindner, A. Krause, S. Trimpe, and D. Baumann, “Gosafeopt: Scalable safe exploration for global optimization of dynamical systems,” *Artificial Intelligence*, vol. 320, p. 103922, 2023.
- [8] D. Widmer, D. Kang, B. Sukhija, J. Hübotter, A. Krause, and S. Coros, “Tuning legged locomotion controllers via safe bayesian optimization,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2444–2464.

- [9] H. Guo, Z. Qi, and X. Liu, “Rectified pessimistic-optimistic learning for stochastic continuum-armed bandit with constraints,” in *Learning for Dynamics and Control Conference*. PMLR, 2023, pp. 1333–1344.
- [10] Unitree Robotics, <https://www.unitree.com/en/go2>.
- [11] I. Mordatch, K. Lowrey, and E. Todorov, “Ensemble-cio: Full-body dynamic motion planning that transfers to physical humanoids,” in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2015, pp. 5307–5314.
- [12] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.
- [13] S. Zhu, A. Kimmel, K. E. Bekris, and A. Boularias, “Model identification via physics engines for improved policy search,” *CoRR*, abs/1710.08893, 2017.
- [14] Y. Sui, A. Gotovos, J. Burdick, and A. Krause, “Safe exploration for optimization with gaussian processes,” in *International conference on machine learning*. PMLR, 2015, pp. 997–1005.
- [15] D. Baumann, A. Marco, M. Turchetta, and S. Trimpe, “Gosafe: Globally optimal safe robot learning,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 4452–4458.
- [16] C. K. Williams and C. E. Rasmussen, *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006, vol. 2.
- [17] S. R. Chowdhury and A. Gopalan, “On kernelized multi-armed bandits,” in *International Conference on Machine Learning*. PMLR, 2017, pp. 844–853.
- [18] Q. Liao, Z. Li, A. Thirugnanam, J. Zeng, and K. Sreenath, “Walking in narrow spaces: Safety-critical locomotion control for quadrupedal robots with duality-based optimization,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 2723–2730.
- [19] C. Li, S. Gupta, S. Rana, V. Nguyen, S. Venkatesh, and A. Shilton, “High dimensional bayesian optimization using dropout,” *arXiv preprint arXiv:1802.05400*, 2018.
- [20] K. Kandasamy, J. Schneider, and B. Póczos, “High dimensional bayesian optimisation and bandits via additive models,” in *International conference on machine learning*. PMLR, 2015, pp. 295–304.
- [21] C.-L. Li, K. Kandasamy, B. Póczos, and J. Schneider, “High dimensional bayesian optimization via restricted projection pursuit models,” in *Artificial Intelligence and Statistics*. PMLR, 2016, pp. 884–892.
- [22] D. Ulmasov, C. Baroukh, B. Chachuat, M. P. Deisenroth, and R. Misener, “Bayesian optimization with dimension scheduling: Application to biological systems,” in *Computer Aided Chemical Engineering*. Elsevier, 2016, vol. 38, pp. 1051–1056.